COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	**CPE010**	**SECTION:**	**CPE21S2**
DATE PERFORMED:	**July 10, 2026**	**DATE SUBMITTED:**	July 15, 2026
NAME:	**Macatangay, Justin Jake B.**	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 **[Basic C++ Program]**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

Sections	**Answer**
Header File Declaration Section	`#include <iostream>`

Global Declaration Section	`// Function prototypes (declared before main for clarity)`
	`void displaySum(int x, int y);  // Displays the sum of two integers`
	`bool isGreater(int a, int b);  // Returns true if a > b`
	`bool logicalOperations(bool p, bool q); // Demonstrates logical operators`

Class Declaration and Method Definition Section	`class Triangle {`
	`private:`
	`  // Private member variables to store angles`
	`  double angleA, angleB, angleC, totalAngle;`
	
	`public:`
	`  // Constructor: initializes angles and computes total`
	`  Triangle(double A, double B, double C) {`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	`  totalAngle = A + B + C;`
	`  }`
	
	`  // Setter method: allows updating angles after object creation`
	`  void setAngles(double A, double B, double C) {`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	`  totalAngle = A + B + C;`
	`  }`
	
	`  // Validates if the angles form a proper triangle`
	`  // Conditions: sum must equal 180, and all angles > 0`
	`  const bool validateTriangle() {`
	`  return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);`
	`  }`

Main Function

```cpp
int main() {
    int num1 = 10, num2 = 20;        // Example integers
    bool val1 = true, val2 = false;  // Example boolean values

    // Call function to display sum of num1 and num2
    displaySum(num1, num2);

    // Check if num1 is greater than num2
    if (isGreater(num1, num2)) {
        std::cout << num1 << " is greater than " << num2 << std::endl;
    } else {
        std::cout << num1 << " is not greater than " << num2 << std::endl;
    }

    // Perform logical operations on val1 and val2
    if (logicalOperations(val1, val2)) {
        std::cout << "Logical operations executed successfully." << std::endl;
    }

    return 0; // End of program
}
```

Method Definition

```cpp
public:
    // Constructor: initializes angles and computes total
    Triangle(double A, double B, double C) {
        angleA = A;
        angleB = B;
        angleC = C;
        totalAngle = A + B + C;
    }

    // Setter method: allows updating angles after object creation
    void setAngles(double A, double B, double C) {
        angleA = A;
        angleB = B;
        angleC = C;
        totalAngle = A + B + C;
    }

    // Validates if the angles form a proper triangle
    // Conditions: sum must equal 180, and all angles > 0
    const bool validateTriangle() {
        return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0
    }
```

```cpp
// Function definition: prints the sum of two integers
void displaySum(int x, int y) {
    std::cout << "Sum of " << x << " and " << y << " is: " << (x + y) << std::endl;
}

// Function definition: returns true if a > b
bool isGreater(int a, int b) {
    return (a > b);
}
```

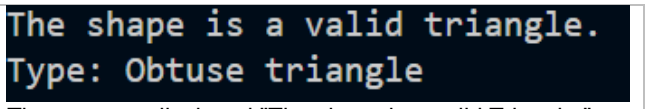
```cpp
// Function definition: demonstrates logical operators
bool logicalOperations(bool p, bool q) {
    // Logical AND
    std::cout << "p AND q: " << (p && q) << std::endl;
    // Logical OR
    std::cout << "p OR q: " << (p || q) << std::endl;
    // XOR (bitwise exclusive OR works fine with bools)
    std::cout << "p XOR q: " << (p ^ q) << std::endl;
    // Logical NOT
    std::cout << "NOT p: " << (!p) << std::endl;
    std::cout << "NOT q: " << (!q) << std::endl;

    // Always returns true (could be improved to return something meaningful)
    return true;
}
```

```cpp
    // Example: 40 + 30 + 110 = 180, valid triangle
    Triangle set1(40, 30, 110);

    // Validate and classify
    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
        std::cout << "Type: " << set1.classifyTriangle() << std::endl;
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

	 The program displayed "The shape is a valid Triangle." because the angles 40°, 30°, and 110° have a total of 200°, which is greater than 180°.. The activity helped me understand how constructors, objects, and member functions work together to validate data in C++..

Lessons Learned Summary
In this exercise, I have realized that I have been relying too much on using using namespace std;. This resulted in me forgetting how to correctly use the standard namespace, including std::cout and std::cin. Without the aid of using namespace std;, I became more accustomed to the right syntax in C++.

Discussion of the Procedure
During the process of executing the procedure, I realized how useful it was to arrange the code using function declarations, main(), and the function definitions. The way in which function definitions were written below main() helped me see clearly how functions can be used. It was also quite impressive how the class had brought about some organization in the code.

Analysis of the Supplementary Activity
Supplementary activity allowed me to use the concept rather than read about it. The first three exercises allowed me to get used to writing and invoking functions, whereas the fourth exercise pushed me to use classes and their member functions. Even though working with Triangle class turned out to be the most difficult part, modifying the class allowed me to understand that the functions within a class can do different things while dealing with the same data.

Conclusion/Feedback
On a whole, I believe that I did well in this task because all the topics were a revision of what we were taught in Programming Logic and Design (PLD). The only aspect that I had difficulty with was using the classes as I am used to the use of classes in Python in our Object-Oriented Programming lessons. Nevertheless, it was good to be able to apply the use of classes in C++, even though the task only revolved around the basics of object-oriented programming.

The best part about this activity was the use of function prototypes, declaration of the functions after the main() function, and calling the functions whenever needed.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1.Create a C++ program to swap the two numbers in different variables.

```cpp
#include <iostream>
int main() {
    int a = 5, b = 10, value;
    value = a;
    a = b;
    b = value;
    std::cout << "After swap: a = " << a << ", b = " << b << std::endl;
    return 0;
}
```

```
After swap: a = 10, b = 5
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```cpp
#include <iostream>
double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9/5 + 32;
}
int main() {
    double kelvin = 300;
    std::cout << kelvin << " K = " << kelvinToFahrenheit(kelvin) << " °F" << std::endl;
    return 0;
}
```

```
300 K = 80.33 °F
```

3. Create a C++ program that has a function that will calculate the distance between two points.

```cpp
#include <iostream>
#include <cmath>
double distance(double x1, double y1, double x2, double y2) {
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
}
int main() {
    std::cout << "Distance: " << distance(2, 3, 7, 8) << std::endl;
    return 0;
}
```

```
Distance: 7.07107
```

4. Modify the code given in ILO B and add the following functions:

```cpp
#include <iostream>

class Triangle {
private:

    double angleA, angleB, angleC, totalAngle;

public:

    Triangle(double A, double B, double C) {
        angleA = A;
        angleB = B;
        angleC = C;
        totalAngle = A + B + C;
    }

    void setAngles(double A, double B, double C) {
        angleA = A;
        angleB = B;
        angleC = C;
        totalAngle = A + B + C;
    }

    const bool validateTriangle() {
        return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);
    }

    std::string classifyTriangle() {
        if (!validateTriangle()) return "Not a valid triangle";

        if (angleA == 90 || angleB == 90 || angleC == 90)
            return "Right triangle";

        else if (angleA > 90 || angleB > 90 || angleC > 90)
            return "Obtuse triangle";

        else
            return "Acute triangle";
    }
};
```

```cpp
int main() {

    Triangle set1(40, 30, 110);

    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
        std::cout << "Type: " << set1.classifyTriangle() << std::endl;
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

```
The shape is a valid triangle.
Type: Obtuse triangle
```

a. A function to compute for the area of a triangle
b. A function to compute for the perimeter of a triangle
c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Microsoft Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used to help and anaylsis some may have Ai on it, but it help me to understand the variables, the function and the flow to how it works.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.